



**UNIVERSIDAD DE
COSTA RICA**
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 5: Reforzamiento Full-Stack con Spring Boot, JPA/Hibernate, HTML5 y CSS3

Resumen

Esta práctica guiada tiene como objetivo reforzar de manera integral el desarrollo de aplicaciones web de pila completa (Full-Stack). Usted integrará la capa de persistencia objeto-relacional (ORM) basada en Spring Data JPA e Hibernate con servicios REST en Spring Boot, y construirá un front-end responsivo y semántico utilizando HTML5 y CSS3.

A lo largo del laboratorio, usted modelará el dominio de alquileres en la base de datos relacional VideoRent, optimizará las consultas asíncronas para evitar problemas de rendimiento N+1, configurará políticas de CORS en controladores REST y consumirá la API desde una interfaz web estilizada mediante el modelo de caja, Flexbox y CSS Grid.

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java 21 (o versión local instalada), SQL Server Developer Edition, SQL Server Management Studio (SSMS), Maven, Navegador Web (Chrome/Firefox), VS Code.

Metas de Aprendizaje:

1. Mapear relaciones de persistencia JPA e implementar consultas optimizadas con `JOIN FETCH` en Spring Data JPA para el módulo de alquileres.
2. Exponer servicios web RESTful seguros y habilitar el intercambio de recursos de origen cruzado (CORS) en Spring Boot.
3. Construir un formulario e interfaz de usuario accesible con marcado semántico HTML5 (`<header>`, `<main>`, `<section>`, `<article>`, `<form>`).
4. Diseñar un maquetado responsivo utilizando CSS Flexbox, CSS Grid Layout, variables CSS y tarjetas dinámicas de estado.

Conceptos Clave (El 'Qué')

1. Integración Full-Stack Desacoplada

Una arquitectura full-stack moderna desacopla la lógica del backend (Spring Boot / JPA) del frontend visual (HTML5 / CSS3 / JavaScript). La comunicación se realiza mediante mensajes JSON asíncronos sobre el protocolo HTTP utilizando la Fetch API.

2. Control de CORS (Cross-Origin Resource Sharing)

Cuando una interfaz web HTML alojada en un origen o servidor local intenta consumir una API REST que se ejecuta en otro puerto o dominio (ej: `http://localhost:8080`), el navegador bloquea la solicitud por seguridad. Es necesario habilitar la anotación `@CrossOrigin` en Spring Boot para permitir el flujo de datos.

3. HTML5 Semántico y Accesibilidad

El uso de etiquetas semánticas (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<form>`) proporciona significado estructural al documento web, mejorando la usabilidad, la usabilidad para lectores de pantalla y la mantenibilidad del código respecto al uso indiscriminado de elementos `<div>`.

4. Layouts Modernos con CSS Grid y Flexbox

- **Flexbox:** Diseñado para la distribución de elementos en una sola dimensión (filas o columnas), ideal para barras de navegación, alineación de botones y formularios.
- **CSS Grid:** Diseñado para maquetados bidireccionales (filas y columnas simultáneas), ideal para la disposición de tableros de control y catálogos de tarjetas (*cards*).

Guía de Base de Datos (SSMS Script)

Para este laboratorio, usted extenderá la base de datos `VideoRent[Carné]_II2026` utilizada en las prácticas anteriores.

1. Abra SQL Server Management Studio (SSMS) y conéctese a su servidor local.
2. Ejecute el siguiente script SQL para crear la tabla `Alquiler` e insertar los datos iniciales de prueba (recuerde sustituir `[Carné]` por su carnet universitario en mayúsculas):

```
USE VideoRent[Carné]_II2026;
GO

-- 1. Crear tabla Alquiler para la gestion de rentas
CREATE TABLE Alquiler (
    alquiler_id INT IDENTITY(1,1) PRIMARY KEY,
    cliente_nombre VARCHAR(100) NOT NULL,
    pelicula_id INT NOT NULL,
    fecha_alquiler DATETIME NOT NULL,
    fecha_devolucion DATETIME NULL,
    monto_diario DECIMAL(10,2) NOT NULL,
    estado VARCHAR(20) NOT NULL,
    CONSTRAINT FK_Alquiler_Pelicula FOREIGN KEY (pelicula_id)
```

```

REFERENCES Pelicula(pelicula_id)
);
GO

-- 2. Insertar registros de semillas iniciales
INSERT INTO Alquiler (cliente_nombre, pelicula_id, fecha_alquiler,
                      fecha_devolucion, monto_diario, estado)
VALUES
('Maria Rodriguez', 1, GETDATE(), NULL, 1500.00, 'ACTIVO'),
('Carlos Gomez', 1, GETDATE()-5, GETDATE()-1, 1500.00, 'DEVUELTO');
GO

```

Sección 1: Capa Backend con Spring Boot, JPA e Hibernate

Sustituya <carnet> por su carnet universitario en minúsculas en las declaraciones de paquetes Java.

1. Entidad JPA (Alquiler.java):

Cree la clase de entidad en el paquete `domain` para mapear la tabla `Alquiler` y su relación `@ManyToOne` con `Pelicula`:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica5/domain/Alquiler.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica5.domain;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "Alquiler")
public class Alquiler {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "alquiler_id")
    private Integer id;

    @Column(name = "cliente_nombre", nullable = false, length = 100)
    private String clienteNombre;
}

```

```

@Column(name = "fecha_alquiler", nullable = false)
private LocalDateTime fechaAlquiler;

@Column(name = "fecha_devolucion")
private LocalDateTime fechaDevolucion;

@Column(name = "monto_diario", nullable = false)
private BigDecimal montoDiario;

@Column(nullable = false, length = 20)
private String estado;

@ManyToOne(fetch = FetchType.EAGER)
@JoinColumn(name = "pelicula_id", nullable = false)
private Pelicula pelicula;

public Alquiler() {}

public Integer getId() { return id; }
public void setId(Integer id) { this.id = id; }

public String getClienteNombre() { return clienteNombre; }
public void setClienteNombre(String c) { this.clienteNombre = c; }

public LocalDateTime getFechaAlquiler() { return fechaAlquiler; }
public void setFechaAlquiler(LocalDateTime f) {
    this.fechaAlquiler = f;
}

public LocalDateTime getFechaDevolucion() { return fechaDevolucion; }
public void setFechaDevolucion(LocalDateTime f) {
    this.fechaDevolucion = f;
}

public BigDecimal getMontoDiario() { return montoDiario; }
public void setMontoDiario(BigDecimal m) { this.montoDiario = m; }

public String getEstado() { return estado; }
public void setEstado(String e) { this.estado = e; }

public Pelicula getPelicula() { return pelicula; }
public void setPelicula(Pelicula p) { this.pelicula = p; }
}

```

2. Repositorio JPA con Optimización (AlquilerRepository.java):

Cree la interfaz en el paquete `data` definiendo una consulta JPQL con `JOIN FETCH` para precargar la película asociada sin generar el problema N+1 Select:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica5/data/AlquilerRepository.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica5.data;

import cr.ac.ucr.paraiso.ie.carnet.practica5.domain.Alquiler;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface AlquilerRepository
    extends JpaRepository<Alquiler, Integer> {

    @Query("SELECT a FROM Alquiler a JOIN FETCH a.pelicula p " +
        "WHERE a.estado = :estado")
    List<Alquiler> findByEstadoConPelicula(String estado);
}
```

3. Capa de Negocio (AlquilerService.java):

Implemente el servicio de negocio en el paquete `business` inyectando el repositorio por constructor y administrando la demarcación transaccional:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica5/business/AlquilerService.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica5.business;

import cr.ac.ucr.paraiso.ie.carnet.practica5.data.AlquilerRepository;
import cr.ac.ucr.paraiso.ie.carnet.practica5.domain.Alquiler;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.List;

@Service
public class AlquilerService {

    private final AlquilerRepository repository;

    public AlquilerService(AlquilerRepository repository) {
        this.repository = repository;
    }
}
```

```

    }

    @Transactional(readOnly = true)
    public List<Alquiler> obtenerAlquileresActivos() {
        return repository.findByEstadoConPelicula("ACTIVO");
    }

    @Transactional
    public Alquiler registrarAlquiler(Alquiler nuevo) {
        nuevo.setEstado("ACTIVO");
        if (nuevo.getFechaAlquiler() == null) {
            nuevo.setFechaAlquiler(LocalDate.now());
        }
        return repository.save(nuevo);
    }
}

```

4. Controlador REST (AlquilerController.java):

Cree el controlador en el paquete controller, habilitando @CrossOrigin para permitir peticiones desde la aplicación web frontal:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica5/controller/AlquilerController.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica5.controller;

import cr.ac.ucr.paraiso.ie.carnet.practica5.business.AlquilerService;
import cr.ac.ucr.paraiso.ie.carnet.practica5.domain.Alquiler;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/alquileres")
@CrossOrigin(origins = "")
public class AlquilerController {

    private final AlquilerService service;

    public AlquilerController(AlquilerService service) {
        this.service = service;
    }

    @GetMapping("/activos")
    public ResponseEntity<List<Alquiler>> getActivos() {

```

```

        return ResponseEntity.ok(service.obtenerAlquileresActivos());
    }

    @PostMapping
    public ResponseEntity<Alquiler> crear(@RequestBody Alquiler nuevo) {
        Alquiler creado = service.registrarAlquiler(nuevo);
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    }
}

```

Sección 2: Capa Frontend con HTML5 Semántico, CSS3 y JavaScript

Cree la carpeta `frontend/` dentro del proyecto para ubicar la interfaz web de usuario.

1. Maquetado HTML5 Semántico (`index.html`):

Construya el marcado estructurado con formularios accesibles y etiquetas semánticas de HTML5:
`frontend/index.html`

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>VideoRent - Gestion de Alquileres</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header class="app-header">
        <h1>VideoRent System</h1>
        <nav>
            <span class="badge-status">API Conectada</span>
        </nav>
    </header>

    <main class="container">
        <section class="card-form">
            <h2>Registrar Nuevo Alquiler</h2>
            <form id="rentalForm">
                <div class="form-group">

```



```

        <label for="cliente">Nombre del Cliente:</label>
        <input type="text" id="cliente" required
        placeholder="Ej: Laura Mora">
    </div>
    <div class="form-group">
        <label for="peliculaId">ID de Película:</label>
        <input type="number" id="peliculaId" required
        placeholder="Ej: 1">
    </div>
    <div class="form-group">
        <label for="monto">Monto Diario (CRC):</label>
        <input type="number" step="0.01" id="monto" required
        placeholder="Ej: 1500.00">
    </div>
    <button type="submit" class="btn-primary">
        Procesar Alquiler
    </button>
</form>
</section>
</div>
<div id="gridContainer" class="rentals-grid">
    <!-- Tarjetas cargadas dinamicamente mediante Fetch API -->
</div>
</section>
</main>

<script src="app.js"></script>
</body>
</html>

```

2. Estilos CSS3 Modernos (styles.css):

Implemente variables CSS, Flexbox para la barra superior y CSS Grid Layout para las tarjetas de catálogo:

frontend/styles.css

```

:root {
    --primary-color: #2563eb;
    --bg-color: #f8fafc;
    --card-bg: #ffffff;
    --text-color: #0f172a;
    --border-color: #cbd5e1;
    --success-color: #16a34a;
}

```

```

}

body {
  font-family: Arial, Helvetica, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-color);
  margin: 0;
  padding: 0;
}

.app-header {
  background-color: var(--primary-color);
  color: white;
  padding: 1rem 2rem;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.badge-status {
  background-color: #1e40af;
  padding: 0.3rem 0.8rem;
  border-radius: 12px;
  font-size: 0.85rem;
}

.container {
  max-width: 1100px;
  margin: 2rem auto;
  padding: 0 1rem;
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 2rem;
}

.card-form, .card-table {
  background: var(--card-bg);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 1.5rem;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
}

.form-group {
  margin-bottom: 1rem;
  display: flex;

```

```

    flex-direction: column;
}

.form-group label {
    font-weight: bold;
    margin-bottom: 0.4rem;
    font-size: 0.9rem;
}

.form-group input {
    padding: 0.6rem;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    font-size: 0.95rem;
}

.btn-primary {
    background-color: var(--primary-color);
    color: white;
    border: none;
    padding: 0.75rem 1.5rem;
    border-radius: 4px;
    font-weight: bold;
    cursor: pointer;
    width: 100%;
}

.btn-primary:hover {
    background-color: #1d4ed8;
}

.rentals-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));
    gap: 1rem;
}

.rental-card {
    border: 1px solid var(--border-color);
    border-radius: 6px;
    padding: 1rem;
    background-color: #f1f5f9;
}

.rental-card h3 {
    margin-top: 0;

```

```

    color: var(--primary-color);
    font-size: 1.1rem;
}

.pill-activo {
    background-color: var(--success-color);
    color: white;
    padding: 0.2rem 0.6rem;
    border-radius: 12px;
    font-size: 0.75rem;
    font-weight: bold;
    display: inline-block;
}

```

3. Lógica Cliente de Integración (app.js):

Programa las solicitudes asíncronas HTTP con `fetch()` y manipulaciones del DOM para consultar y enviar datos a Spring Boot:

frontend/app.js

```

const API_URL = 'http://localhost:8080/api/alquileres';

document.addEventListener('DOMContentLoaded', () => {
    cargarAlquileres();

    document.getElementById('rentalForm')
        .addEventListener('submit', guardarAlquiler);
});

async function cargarAlquileres() {
    try {
        const resp = await fetch(`${API_URL}/activos`);
        if (!resp.ok) throw new Error('Error en el servidor');
        const data = await resp.json();
        renderGrid(data);
    } catch (err) {
        console.error('Error al obtener alquileres:', err);
    }
}

function renderGrid(alquileres) {
    const container = document.getElementById('gridContainer');
    container.innerHTML = '';

    if (alquileres.length === 0) {

```

```

        container.innerHTML = '<p>No hay alquileres activos.</p>';
        return;
    }

    alquileres.forEach(a => {
        const card = document.createElement('article');
        card.className = 'rental-card';
        const peliculaTitulo = a.pelicula ? a.pelicula.titulo : 'N/A';
        card.innerHTML = `
            <h3>${a.clienteNombre}</h3>
            <p><strong>Película:</strong> ${peliculaTitulo}</p>
            <p><strong>Monto/día:</strong> CRC ${a.montoDiario}</p>
            <span class="pill-activo">${a.estado}</span>
        `;
        container.appendChild(card);
    });
}

async function guardarAlquiler(e) {
    e.preventDefault();
    const payload = {
        clienteNombre: document.getElementById('cliente').value,
        pelicula: { id: parseInt(document.getElementById('peliculaId').value) },
        montoDiario: parseFloat(document.getElementById('monto').value)
    };

    try {
        const resp = await fetch(API_URL, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });

        if (resp.ok) {
            document.getElementById('rentalForm').reset();
            cargarAlquileres();
        } else {
            alert('Fallo al registrar alquiler.');
```

Taller de Depuración (El Error Común)

El objetivo de este taller es identificar y solucionar un bloqueo por política de CORS (*Cross-Origin Resource Sharing*).

Paso 1: Provocar el Error

- [] Comente temporalmente la anotación `@CrossOrigin(origins = "*")` en la clase `AlquilerController.java`.
- [] Ejecute la aplicación Spring Boot mediante `mvn spring-boot:run`.
- [] Abra el archivo `frontend/index.html` en el navegador web e intente cargar la lista de alquileres.
- [] Abra la consola de herramientas de desarrollador (F12 -> *Console*). Usted observará un mensaje de error similar a:
`Access to fetch at 'http://localhost:8080/api/alquileres/activos' from origin 'null' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.`

Paso 2: Diagnóstico

El navegador bloquea la lectura de la respuesta JSON porque la aplicación frontal (`file://` o servidor estático en puerto distinto) y la API REST (`http://localhost:8080`) pertenecen a orígenes de red diferentes. La API de Spring Boot no incluyó las cabeceras de respuesta HTTP acordes (`Access-Control-Allow-Origin`).

Paso 3: Resolución del Error

- [] Restaure la anotación `@CrossOrigin(origins = "*")` sobre el controlador `AlquilerController.java` o configure un filtro global de CORS en su paquete de configuración de Spring Boot:

```
package cr.ac.ucr.paraíso.ie.carnet.practica5.config;  
  
import org.springframework.context.annotation.Configuration;  
import org.springframework.web.servlet.config.annotation.CorsRegistry;
```

```
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

- [] Recargue la aplicación en el navegador y verifique en la pestaña *Network* que la solicitud GET retorne el código de estado HTTP 200 OK con los datos de los alquileres.

Parte 3: Reto Autónomo (Sin Solución)

Debe desarrollar de forma autónoma la funcionalidad de devolución de películas alquiladas.

Requerimientos Backend:

- [] En `AlquilerRepository.java`, cree un método para buscar alquileres activos por el ID de la película.
- [] En `AlquilerService.java`, agregue un método `@Transactional` denominado `procesarDevolucion(Integer alquilerId)` que cambie el estado del alquiler a `"DEVUELTO"` y registre la fecha actual en `fechaDevolucion`.
- [] En `AlquilerController.java`, exponga un endpoint de tipo `PATCH` o `PUT` en `/api/alquileres/{id}/devolucion`.

Requerimientos Frontend:

- [] Modifique el archivo `styles.css` agregando una clase `.pill-devuelto` con un color distintivo (ej. gris o azul secundario #64748b).

- [] En `app.js`, agregue un botón "Devolver" dentro de cada tarjeta de alquiler activo.
- [] Al hacer clic en el botón "Devolver", envíe una petición asíncrona mediante `fetch()` al endpoint de devolución, actualice la interfaz y verifique que la tarjeta se remueva o cambie visualmente su estado.